

Amazon Aurora MySQL Provisioned Cluster Setup

To Begin with the Lab

Summary of the Lab

In this lab, an **Amazon Aurora MySQL** provisioned database cluster was created from the **Amazon RDS** console instead of using serverless. The cluster was configured using **Standard create**, with a version that supports the global database feature and Aurora Serverless v2 for future use. A production-style setup was chosen with a provisioned burstable instance class, an additional Aurora replica in another Availability Zone, private VPC networking, and no public access. The configuration also included options such as automated backups, encryption, Performance Insights, and optional failover priority settings. After creation, the cluster was verified to contain a writer instance, a reader instance, a writer endpoint, and a reader endpoint for application access.

Understanding Amazon Aurora Provisioned Clusters

An **Amazon Aurora provisioned cluster** is a database deployment where you choose and manage the compute capacity for each instance in the cluster. It exists so you can run a high-availability database with a writer instance for all writes and one or more reader instances for read scaling and failover support. Aurora clusters also provide separate writer and reader endpoints, which makes application routing easier because write traffic and read traffic can be sent to different targets. This is useful for production workloads that need strong availability, controlled performance, and the option to grow later into features such as global databases or additional replicas.

Example:

A web application stores orders in Aurora MySQL. The application sends INSERT and UPDATE operations to the writer endpoint, while reporting dashboards send SELECT queries to the reader endpoint. If the writer fails, Aurora can fail over to a replica in another Availability Zone and keep the database available.

Lab Steps

Step 1 – Start the Aurora Cluster Creation

- Sign in to the AWS Management Console.
- Open **Amazon RDS**.
- Click **Create database**.
- Select **Standard create**.
- Choose **Aurora MySQL compatible**.

Create database [Info](#)

Availability zone group [Info](#)
 Availability zone group
 Select the availability zone where you want to create your database.
 US East (N. Virginia)

Engine options
Engine type [Info](#)
☒ Aurora (MySQL Compatible)
☐ Aurora (PostgreSQL Compatible)
☐ MySQL
☐ PostgreSQL
☐ MariaDB
☐ Oracle
☐ Microsoft SQL Server
☐ IBM Db2

Choose a database creation method
☒ Full configuration
☐ Easy create

- Choose Database creation method as **Full configuration**.
- Choose **Production** as the template.
- Choose **Provisioned Cluster**.
- And for Lab purpose, I have taken the lowest instance class but for updated support, we have to choose **db.t3.medium** so that it can support latest versions.

Choose a database creation method
☒ Full configuration
☐ Easy create

Templates
 Choose a sample template to meet your use case.
☒ Production
☐ Dev/Test

Cluster scalability type [Info](#)
☐ Aurora serverless
☒ Provisioned

Type of provisioned configuration [Info](#)
 Choose a configuration type that aligns with your workload requirements.
☐ Memory optimized classes (includes r classes)
☒ Burstable classes (includes t classes)
 db.t3.medium

Step 2 – Configure the Database Cluster

- Choose the compatible Engine Version based on your MySQL Version.
- Enter a database cluster identifier.
- Use the default master username.
- Tick the **Enable RDS Extended Support** option.

Settings

Engine version
View the engine versions based on your selected database and cluster scalability type.
Aurora MySQL 3.12.0 (compatible with MySQL 8.0.44)

☐ Enable RDS Extended Support
Amazon RDS Extended Support is a paid offering. By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the [Amazon Aurora documentation](#).

DB cluster identifier
Type a name for your DB cluster. The name must be unique across all DB clusters owned by your AWS account in the current AWS Region.
database-2
The DB cluster identifier is case-insensitive, but is stored as all lowercase (as in "mydbcluster"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Credentials Settings

Master username
Type a login ID for the master user of your DB instance.
admin
1 to 32 alphanumeric characters. The first character must be a letter.

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.
☐ Managed in AWS Secrets Manager
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.
☒ Self managed
Generate your own password or have RDS create a password that you manage.

☐ Auto generate password
Amazon RDS can generate a password for you, or you can specify your own password.

Master password
[Masked password]

- I will choose **Self Managed** option.
- Set the master password and choose the **Password authentication** method.

☐ Managed in AWS Secrets Manager
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ Self managed
Generate your own password or have RDS create a password that you manage.

☐ Auto generate password
Amazon RDS can generate a password for you, or you can specify your own password.

Master password
[Masked password]
Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / * @
Password strength: Weak

Confirm master password
[Masked password]

Additional credentials settings

Database authentication
Manage your database user's credentials. Password authentication is always active for your database engine. You can turn on additional authentication methods for your database below.

Database authentication options
☒ Password authentication
Authenticates using database passwords.
☐ Password and IAM database authentication
Authenticates using the database password and user credentials through AWS IAM users and roles.

Cluster storage configuration
Choose the storage configuration for the Aurora DB cluster that best fits your application's price predictability and price performance needs.

Configuration options
Database instance, storage, and I/O charges vary depending on the configuration. [Learn more](#)

- Choose **Aurora standard** storage.
- Create an Aurora replica in a different Availability Zone for failover support.

Cluster storage configuration [Info](#)

Choose the storage configuration for the Aurora DB cluster that best fits your application's price predictability and price performance needs.

Configuration options

Database instance, storage, and I/O charges vary depending on the configuration. [Learn more](#)

☐ Aurora I/O-Optimized

- Predictable pricing for all applications. Improved price performance for I/O-intensive applications (I/O costs >25% of total database costs).
- No additional charges for read/write I/O operations. DB instance and storage prices include I/O usage.

☒ **Aurora Standard**

- Cost-effective pricing for many applications with moderate I/O usage (I/O costs <25% of total database costs).
- Pay-per-request I/O charges apply. DB instance and storage prices don't include I/O usage.

Availability & durability

Multi-AZ deployment [Info](#)

☒ **Create an Aurora Replica or Reader node in a different AZ (recommended for scaled availability)**

☐ Don't create an Aurora Replica

Connectivity [Info](#)

Compute resource

Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

☒ **Don't connect to an EC2 compute resource**

Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.

☐ Connect to an EC2 compute resource

Set up a connection to an EC2 compute resource for this database.

Network type [Info](#)

To use dual-stack mode, make sure that you associate an IPv6 CIDR block with a subnet in the VPC you specify.

Step 3 – Configure Connectivity and Security

- Select the VPC where the cluster will be created.
- Do not connect the cluster to an EC2 compute resource automatically.
- Select the VPC if you have created it but I will go with Default values here.
- Set **Public access** to **No** for lab purpose.

Connectivity [Info](#)

Compute resource

Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

☒ **Don't connect to an EC2 compute resource**

Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.

☐ Connect to an EC2 compute resource

Set up a connection to an EC2 compute resource for this database.

Network type [Info](#)

To use dual-stack mode, make sure that you associate an IPv6 CIDR block with a subnet in the VPC you specify.

☒ **IPv4**

Your resources can communicate only over the IPv4 addressing protocol.

☐ Dual-stack mode

Your resources can communicate over IPv4, IPv6, or both.

Virtual private cloud (VPC) [Info](#)

Choose the VPC. The VPC defines the virtual networking environment for this DB cluster.

After a database is created, you can't change its VPC. Only VPCs with a corresponding DB subnet group are listed.

DB subnet group [Info](#)

Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB cluster can use in the VPC that you selected.

Public access [Info](#)

☐ Yes

RDS assigns a public IP address to the cluster. Amazon EC2 instances and other resources outside of the VPC can also connect to your cluster. Resources inside the VPC can also connect to the cluster. Choose one or more VPC security groups that specify which resources can connect to the cluster.

☒ **No**

You can't assign a public IP address to the cluster. Only Amazon EC2 instances and other resources inside the VPC can connect to your cluster. Choose one or more VPC security groups that specify which resources can connect to the cluster.

- Select the existing security group if created otherwise go with **Default**.
- Leave **RDS Proxy** disabled for this lab.

aws [Search] [Alt+S] United States (N. Virginia) cloudriteshk (463646775279) cf-shashank1

Aurora and RDS > Databases > Create database

☒ No
RDS doesn't assign a public IP address to the cluster. Only Amazon EC2 instances and other resources inside the VPC can connect to your cluster. Choose one or more VPC security groups that specify which resources can connect to the cluster.

VPC security group (firewall) [Info](#)
Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

☒ Choose existing ☐ Create new
Choose existing VPC security groups Create new VPC security group

Existing VPC security groups
Choose one or more options

default X

RDS Proxy
RDS Proxy is a highly available database proxy that improves application scalability, resiliency, and security.

☐ Create an RDS Proxy [Info](#)
RDS Proxy requires an IAM role and a Secrets Manager secret for the proxy. RDS Proxy has additional costs. For more information, see [Amazon RDS Proxy pricing](#).

Certificate authority - optional [Info](#)
Using a server certificate provides an extra layer of security by validating that the connection is being made to an Amazon database. It does so by checking the server certificate that is automatically installed on all databases that you provision.

rds-ca-rsa2048-g1 (default)
Expiry: May 26, 2061

If you don't select a certificate authority, RDS chooses one for you.

► Additional configuration

Tags - optional
A tag consists of a case-sensitive key-value pair.

- Keep Performance Insights enabled.

aws [Search] [Alt+S] United States (N. Virginia) cloudriteshk (463646775279) cf-shashank1

Aurora and RDS > Databases > Create database

Monitoring [Info](#)
Choose monitoring tools for this database. Database Insights provides a combined view of Performance Insights and Enhanced Monitoring for your fleet of databases.

☐ Database Insights - Advanced
Retains 15 months of performance history
Fleet-level monitoring
Integration with CloudWatch Application Signals

☒ Database Insights - Standard

CloudWatch Database Insights pricing is separate from RDS monthly estimates. See [Amazon CloudWatch pricing](#).

Performance Insights
☒ Enable Performance Insights
With Performance Insights enabled, you can visualize the database load on your Amazon RDS DB instance load and filter the load by waits, SQL statements, hosts, or users.

Retention period
7 days

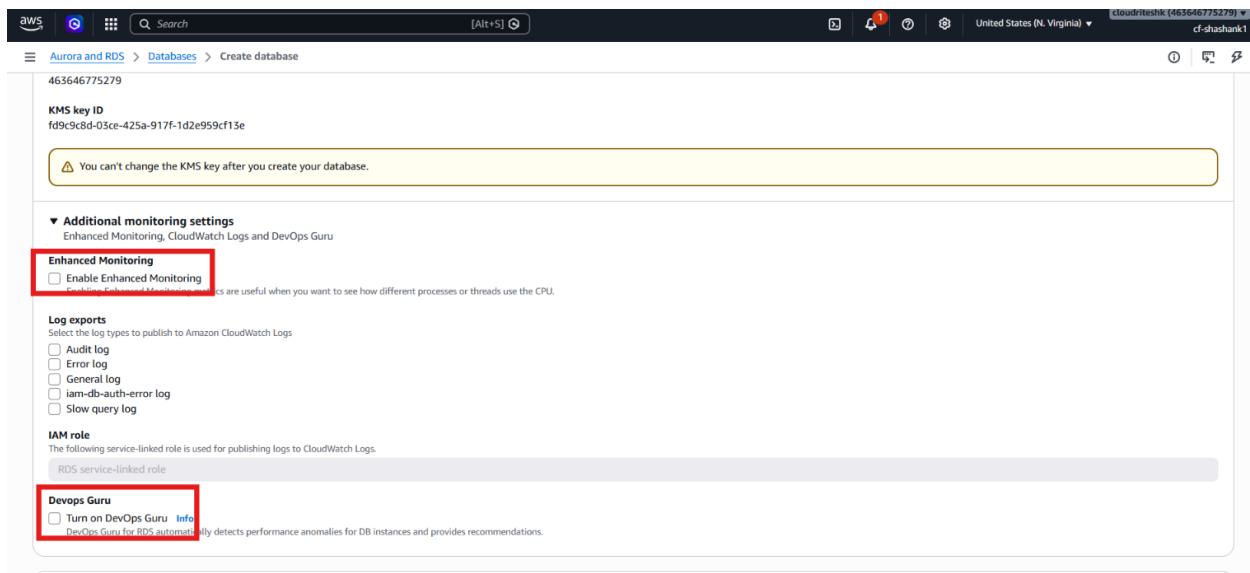
AWS KMS key [Info](#)
Q am:aws:kms:us-east-1:463646775279:key/fd9c9c8d-03ce-425a-917f-1d2e959cf13e X
Example: am:aws:kms:<region>:<accountID>:key/<key-id>

Alias
(default) aws/rds

Account
463646775279

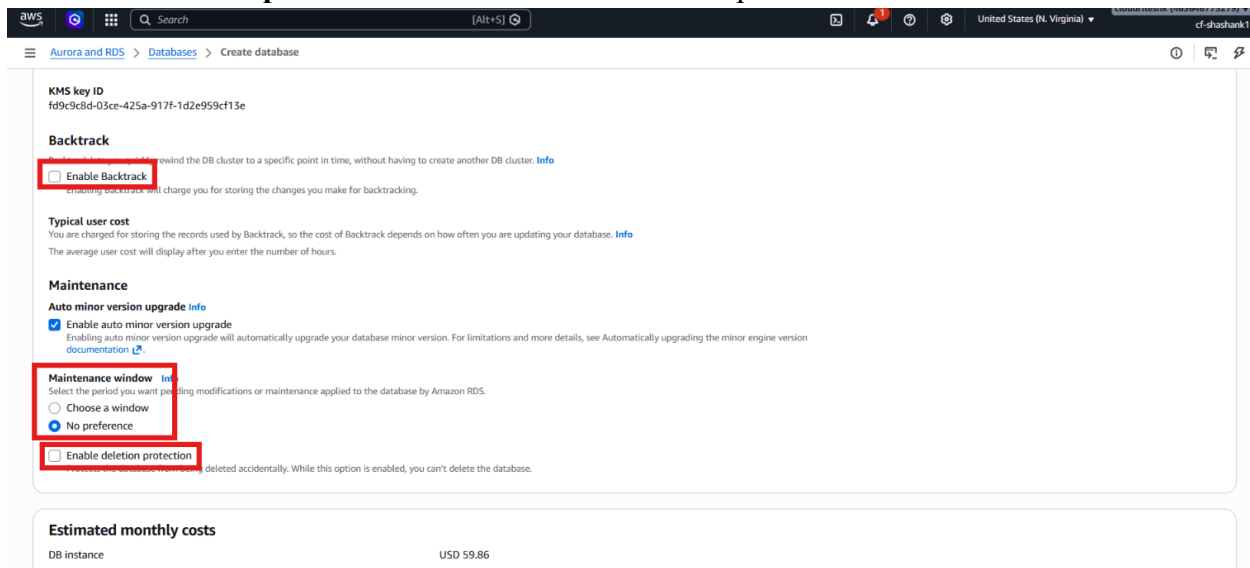
KMS key ID
fd9c9c8d-03ce-425a-917f-1d2e959cf13e

- Disable Enhanced Monitoring and Devops Guru only for lab purpose otherwise they will be automatically enabled.



Step 4 – Configure Advanced Options

- Keep **Backtrack** disabled for this lab.
- Set the maintenance window if needed.
- Leave **Deletion protection** disabled for easier cleanup.



- Click **Create database**.

Step 5 – Wait for Cluster Creation

- Wait while Aurora creates the cluster.
- Confirm that the cluster status becomes available.
- Verify that the cluster contains one writer instance and one reader instance.
- Confirm that the writer handles write operations.
- Confirm that the reader is available for read operations.

Step 6 – Verify Endpoints and Cluster Details

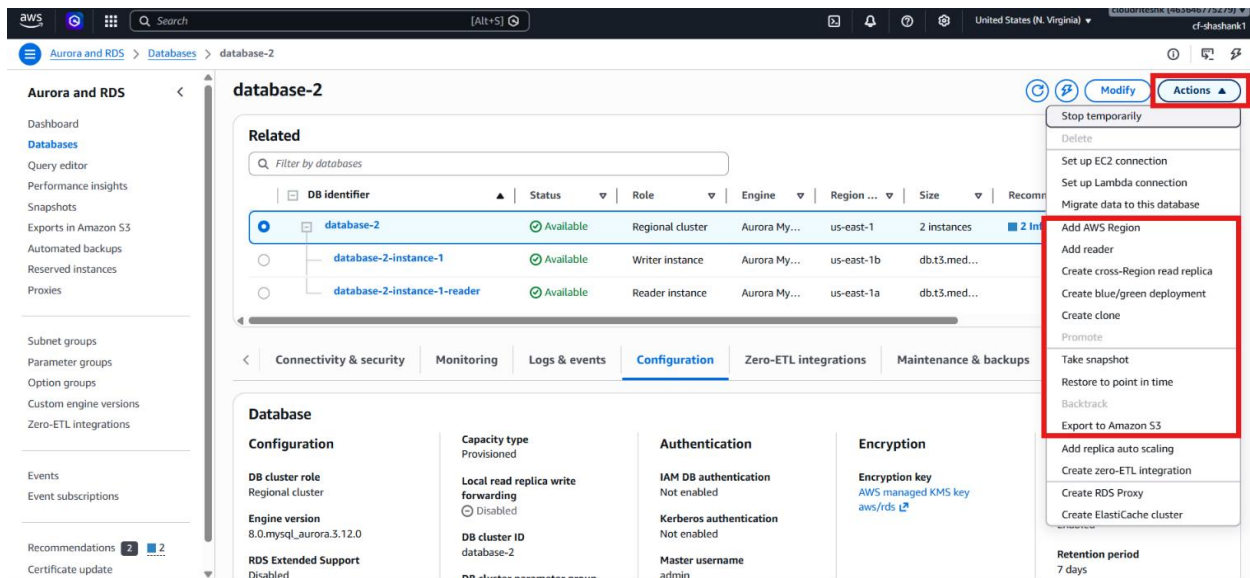
- Open the cluster details page.
- Scroll to **Connectivity and security**.
- Check the **Configuration** section.

The screenshot shows the AWS Aurora and RDS console. The left sidebar contains navigation links for Aurora and RDS, including Dashboard, Databases, Query editor, Performance insights, Snapshots, Exports in Amazon S3, Automated backups, Reserved instances, Proxies, Subnet groups, Parameter groups, Option groups, Custom engine versions, Zero-ETL integrations, Events, Event subscriptions, Recommendations, and Certificate update. The main content area displays the details for a database cluster named 'database-2'. The cluster is in the 'Available' state and is a 'Regional cluster' in the 'us-east-1' region. It has 2 instances: 'database-2-instance-1' (Writer instance) and 'database-2-instance-1-reader' (Reader instance). The 'Configuration' tab is selected, showing various settings: DB cluster role (Regional cluster), Engine version (8.0.mysql_aurora.3.12.0), RDS Extended Support (Disabled), Resource ID (cluster-KHOFNWSW7QTMVS7KCATUKQBX KQ), Cluster storage configuration (Aurora Standard), Capacity type (Provisioned), Local read replica write forwarding (Disabled), DB cluster ID (database-2), DB cluster parameter group (default.aurora-mysql8.0), Deletion protection (Disabled), Limitless Database (Disabled), Authentication (IAM DB authentication: Not enabled, Kerberos authentication: Not enabled, Master username: admin, Master password: ****), Encryption (Encryption key: AWS managed KMS key), Monitoring (Monitoring type: Database Insights - Standard, Performance Insights: Enabled, Retention period: 7 days, AWS KMS key: aws/rds), and Enhanced Monitoring (Disabled). The 'Availability' section shows 'Multi-AZ' and '2 Zones'.

- Confirm that the cluster is provisioned and not multi-master.
- Confirm that the cluster spans two Availability Zones.

Step 7 – Review Post-Creation Actions

- Open **Actions** on the cluster page.
- Review the available actions:
 - Add a reader instance
 - Create a read replica in another Region
 - Take an on-demand snapshot
 - Create a clone
 - Restore to a point in time
- Note that **Backtrack** remains unavailable unless enabled earlier.



Verification

- The **Amazon Aurora MySQL** cluster was created successfully.
- The cluster used the **provisioned** deployment model.
- A writer instance and a reader instance were created.
- The writer endpoint and reader endpoint were available.
- The cluster was spread across two Availability Zones.
- Advanced options such as backups, encryption, and failover priority were reviewed successfully.